TECHNICAL TRAINING STORYBOARD SPECIFICATION

Project Name: GitHub Actions CI/CD Fundamentals

Target Audience: Junior Developers, New Engineering Onboarding

Format: Interactive Self-Service e-Learning (SCORM / xAPI Compliant)

Version: 1.1

Owner: Developer Enablement & Technical Learning Team

1. Executive Summary & Strategy Blueprint

Training Philosophy

Standard documentation teaches developers *what* features exist, but leaves them stranded when a pipeline fails in production. This course is built to drive independent problem-solving. Through safe-to-fail diagnostic simulations, developers build, break, and fix pipelines in a sandbox before touched real production environments.

Key Documentation Gaps Closed

- **The YAML Translation Barrier:** Translates abstract key-value syntax into direct operational mental models.
- **Troubleshooting Friction:** Replaces fragmented error search queries with a structured 3-step diagnostic flow.
- **Directory Structural Errors:** Directly highlights the critical .github/workflows/ path requirements, which are frequently missed by beginners.

2. Global Instructional Design Standards

- **Visual Theme:** Developer Dark Mode (Slate backgrounds, crisp terminal typography, neon green validation status, electric red failure status).
- **Accessibility (Section 508):** High-contrast ratios (minimum 4.5:1), fully screen-reader accessible headers, keyboard navigation fallback, and explicit alternative text for every visual layout.
- **Tone:** Professional, direct, action-oriented, and encouraging.

3. Slide-by-Slide Storyboard Specifications

Slide 1: Course Entry (The Hook)

Element	Specification
Visual Layout	Deep slate background. A dynamic terminal interface displays on the left showing simulated code execution. In the center, a glowing branch icon animates to show: **Code → Build → Test → Release**. An inviting button reads [Initialize Pipeline].
On-Screen Text	**Title:** GitHub Actions CI/CD Fundamentals **Subtitle:** Build absolute confidence into every single code change.
Audio Script	*"Welcome, team! In this training module, we are stripping away the buzzwords and building a continuous validation pipeline from the ground up. You will learn to construct, deploy, and debug your automated workflows autonomously."*
Learner Interaction	Click the glowing [Initialize Pipeline] button.
Variables / State	Linear progression. Unlocks next slide.

Slide 2: Contextual Alignment (The "Why")

Element	Specification
Visual Layout	A side-by-side comparative split layout. **Left Panel (Red-accented):** Icon of a developer running terminal checks manually. A badge displays Risk Profile: High. **Right Panel (Green-accented):** Clean, automated workflow executing containers in parallel. A badge displays Risk Profile: Low.
On-Screen Text	**Header:** Continuous Confidence **Left Bullet points:** Manual checks invite human error; Environment drift ("Worked on my machine!"); Release anxiety blocks velocity. **Right Bullet points:** Automatic verification on push; Isolated virtual environments; Instant validation loops.

Audio Script	*"Relying on manual checklist verification leaves our production environments exposed to human error. Automatic integration pipelines run isolated tests with every push, turning release anxiety into a thing of the past."*
Learner Interaction	Hover hotspots over each panel to reveal comparative team velocity charts.
Variables / State	Hover triggers logged in state metrics. Slide2_Completed sets to True once both panels have been hovered over.

Slide 3: Interactive Architecture Setup

Element	Specification
Visual Layout	An interactive schematic flowchart with 4 clickable nodal components: 1. **Webhook Trigger** 2. **GitHub Orchestrator** 3. **Hosted Runner** 4. **Step Execution**
On-Screen Text	**Header:** The Core Pipeline Architecture **Instruction:** *Click each component in sequence to trace the pipeline execution journey.*
Audio Script	*"GitHub Actions orchestrates container infrastructure on demand. Let's trace how a basic push signal activates an environment, spins up a fresh runner, and processes commands."*
Learner Interaction	User clicks the 4 components in chronological order (1 to 4).
Variables / State	Track variable: Viewed_Component_Count (Increments +1 per click). Next button is hidden until Viewed_Component_Count = 4.
Actionable Feedback	If a learner clicks components out of chronological sequence, display prompt: *"The pipeline follows a sequential flow. Start with Step 1 (Trigger) to see how the runner gets spun up!"*

Slide 4: Detailed Workflow Blueprint (The Anatomy)

Element	Specification
Visual Layout	Split-panel layout. **Left Panel:** Interactive YAML code block editor. Specific sections highlight and glow on click. **Right Panel:** Descriptive breakout drawer containing definition details.
On-Screen Text	**Header:** Anatomy of a Workflow Config **YAML displayed:** name: Node.js CI on: [push] jobs: build: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4
Audio Script	*"Pipelines are constructed using highly organized YAML configuration files. Don't let the syntax overwhelm you; we are simply declaring the trigger event, provisioning a system runner, and mapping execution steps."*
Learner Interaction	Clicking highlighted regions in the code (e.g. clicking on: [push] opens the "Trigger" panel on the right).
Variables / State	Tracks clicks on elements: Name_Clicked, Trigger_Clicked, Jobs_Clicked, Steps_Clicked. All 4 must turn True to unlock progress.

Slide 5: Scenario-Based Diagnosis (Troubleshooting Practice)

Element	Specification
Visual Layout	Simulated GitHub Actions run console. A massive red alert icon displays beside a failed step named Run Linting Suite. Live-scrolling mock error log outputs on a terminal window.

On-Screen Text	**Question:** The Pipeline Failed. What Should You Investigate First? **Option A:** Rewrite the entire workflow configuration file. **Option B:** Review the diagnostic step log output for the failed block. **Option C:** Immediately escalate the ticket to your senior developer.
Audio Script	*"A critical build run failed and threw a red exit code flag. Before rushing to make changes or escalating, let's look at where we can find the exact failure trace."*
Learner Interaction	Multiple choice radio-button selection.
Variables / State	Variable v_debug_choice stores user selection for tracking.
Actionable Feedback	**Select A:** *"Not yet! Rewriting clean files from scratch wastes valuable sprint cycles and ignores root causes."* **Select B (Correct):** *"Outstanding! The diagnostic logs contain the exact line-by-line step failure trace."* **Select C:** *"Try self-service diagnostic checks first. Checking the error output empowers you to solve things independently."*

Slide 6: Repair Workshop (Hands-On Simulation)

Element	Specification
Visual Layout	Simulated VS Code workspace showcasing an incomplete pipeline file. On the right, three draggable logic blocks are placed in a sandbox drawer.
On-Screen Text	**Header:** Repair the Pipeline Code **Instruction:** *Drag the missing dependency step to ensure your python application tests can run successfully without failing.*
Audio Script	*"The majority of integration run failures are caused by missing or unconfigured dependencies. Drag and drop the correct setup step into our workflow timeline."*

Learner Interaction	Drag and drop interaction.
Options Provided	**Block A:** run: python app.py **Block B (Correct):** run: pip install -r requirements.txt **Block C:** run: git commit
Actionable Feedback	**Drop A:** Block snaps back. *"If we run the app before resolving dependencies, the engine will crash. Find the install step first!"* **Drop B (Correct):** Highlights code window in green, locks block in, and displays: *"Perfect! Installing requirements guarantees all libraries are fully loaded before executing the test script."* **Drop C:** Block snaps back. *"Committing inside a running pipeline runner creates recursion loops. Try again!"*

Slide 7: Location Validation (Knowledge Check)

Element	Specification
Visual Layout	File system tree mock showing directory structures: .github/workflows/, .github/workflows/ci.yml, and root folders.
On-Screen Text	**Question:** You commit your new configuration file, but absolutely nothing runs when you push your changes. What is the most likely error? **Option A:** The file was saved outside the .github/workflows folder. **Option B:** The repository size is too large for GitHub to process. **Option C:** The code file was written in Python instead of JavaScript.
Audio Script	*"The orchestration engines at GitHub only monitor a specific directory path. If your configuration isn't placed correctly, GitHub Actions will miss it completely."*
Learner Interaction	Multiple-choice single response selection.
Actionable Feedback	**Option A (Correct):** *"Exactly. Workflows must exist inside the .github/workflows folder at the absolute root of your project directory structure."*

Option B: *"Incorrect. Repository storage size has no bearing on workflow parsing."*

Option C: *"Incorrect. Workflows are language-agnostic and work seamlessly with Python, JS, C#, and more."*

Slide 8: Independent Lab (Proficiency Lab)

Element	Specification
Visual Layout	Split terminal/code window. A sequence checklist showing accomplishments to reach: 1. Directory Setup, 2. Code YAML, 3. Git Push, 4. Status Check.
On-Screen Text	**Header:** Build Your First Configuration **Task:** Complete a simulated mock pipeline run by executing commands: create folder, populate a basic workflow file, push branch, and inspect console outcome.
Audio Script	*"It is time to put your skills to the test. Step through the manual sequence of initializing, configuring, committing, and validating a fresh runner process."*
Success Criteria	Learner must successfully perform the simulated tasks in sequence to pass. Terminal returns a passing indicator code trace.

Slide 9: Module Complete (The Resource Hub)

Element	Specification
Visual Layout	Elegant workspace completion screen. Quick links to reference sheets, cheat guides, enterprise templates, and a PDF version of the *CI/CD Troubleshooting pocket-guide*.
On-Screen Text	**Header:** Continue Your CI/CD Journey **Subtitle:** You have unlocked structural mastery over modern deployment pipelines.
Audio Script	*"Congratulations, developer! You are now equipped to navigate, construct, and debug core automation systems safely and efficiently. Download your quick-reference sheet and go configure some workflows!"*
Learner Interaction	Click and download resource links. Click [Finalize and Log Completion].

4. Maintenance & Evolution Protocol

Course Review Frequency

This specification is audited **quarterly** by the Developer Enablement & Technical Learning Team.

Change Log

Version	Date	Changes Made	Primary Owner
1.0	April 14, 2025	Initial Draft and Core Architecture Mapping	Developer Enablement
1.1	July 10, 2026	Upgraded to feature Scenario-Based Diagnostic and repair simulations	Instructional Design Lead